# Merge Conflict 실습 Hands-on

이 저장소는 Git Merge Conflict를 직접 체험해보는 실습용입니다. 아래 순서대로 따라 하면서 충돌 해결을 연습해 봅시다.

## 실습 목표

- Fast-forward merge를 경험한다
- 3-way merge를 경험한다
- Merge conflict를 **직접 발생시키고** 해결한다
- `git merge --abort` 로 병합을 취소해본다
- Squash merge를 경험한다

---

## 실습 1: Fast-Forward Merge

Fast-forward는 main에 새 커밋이 없을 때 발생합니다.

```
# 1. feature-ff 브랜치 만들기
git switch -c feature-ff

# 2. hello.py에 한 줄 추가
echo 'print("from feature-ff")' >> src/hello.py

# 3. 커밋
git add src/hello.py
git commit -m "feat: add feature-ff greeting"

# 4. main으로 돌아가서 병합
git switch main
git merge feature-ff

# 5. 결과 확인 — Fast-forward 메시지가 나와야 함
git log --oneline --graph
```

## 실습 2: 충돌 발생시키고 해결하기 (핵심!)

같은 파일의 같은 줄을 양쪽에서 다르게 수정해서 충돌을 만듭니다.

### Step 1: main에서 수정

```
git switch main

# src/greeting.py의 print_greeting 함수를 수정하세요
# 기존: print("Hello!")
# 수정: print("안녕하세요!")
```

### Step 2: feature-conflict 브랜치 만들기

```
# main의 현재 상태에서 브랜치 생성
git switch -c feature-conflict
```

### Step 3: feature-conflict에서 같은 함수를 다르게 수정

```
# src/greeting.py의 같은 print_greeting 함수를 수정하세요
# 기존: print("Hello!")
# 수정: print("Hello, World!")
```

### Step 4: feature-conflict 커밋

```
git add src/greeting.py
git commit -m "feat: change greeting to English"
```

### Step 5: main으로 돌아가서 main도 커밋

```
git switch main

# main에서도 같은 함수를 수정 (이미 Step 1에서 수정했으면 커밋만)
git add src/greeting.py
git commit -m "feat: change greeting to Korean"
```

### Step 6: 병합 시도 → 충돌 발생!

```
git merge feature-conflict
# CONFLICT (content): Merge conflict in src/greeting.py
```

### Step 7: 충돌 해결

```
# 1. 파일 열기
code src/greeting.py   # 또는 에디터로 열기

# 2. 충돌 마커 확인
# <<<<<<< HEAD        ← main의 내용 시작
#     print("안녕하세요!")
# =======             ← 구분선
#     print("Hello, World!")
# >>>>>>> feature-conflict  ← feature의 내용 끝

# 3. 원하는 코드만 남기기 (마커는 지우기)
# 예시: 둘 다 살리기
#     print("안녕하세요!")
#     print("Hello, World!")

# 4. 저장 후 add & commit
git add src/greeting.py
git commit -m "merge: resolve greeting conflict"
```

## 실습 3: 병합 취소하기 (abort)

충돌이 너무 복잡하면 취소할 수 있습니다.

```
# 1. conflict-abort 브랜치 만들기
git switch main
git switch -c conflict-abort

# 2. src/utils.py의 get_config 함수 수정 후 커밋
# (자유롭게 수정)

# 3. main으로 돌아가서 같은 함수 수정 후 커밋
git switch main
# (같은 함수 다르게 수정)

# 4. 병합 시도
git merge conflict-abort
# 충돌 발생!

# 5. 포기하고 취소
git merge --abort
```

```
# 6. 깨끗한 상태로 돌아온 것 확인
git status
```

## 실습 4: Squash Merge

여러 커밋을 하나로 합쳐서 병합합니다.

```
# 1. squash 브랜치 만들기
git switch -c feature-squash

# 2. src/calculator.py를 여러 번에 걸쳐 수정하며 커밋
# 첫 번째 커밋
echo "# TODO: add more operations" >> src/calculator.py
git add src/calculator.py
git commit -m "wip: start calculator"

# 두 번째 커밋
# (파일 수정)
git add src/calculator.py
git commit -m "wip: add functions"

# 세 번째 커밋
# (파일 수정)
git add src/calculator.py
git commit -m "feat: complete calculator"

# 3. main에서 squash merge
git switch main
git merge --squash feature-squash
git commit -m "feat: calculator 기능 추가"

# 4. 결과 확인 — 커밋 하나만 생긴 것을 확인
git log --oneline
```

## 실습 5: rebase로 충돌 해결

```
# 1. rebase-practice 브랜치 만들기
git switch -c rebase-practice

# 2. src/hello.py 수정 후 커밋
# 3. main으로 돌아가서 src/hello.py 수정 후 커밋
# 4. rebase로 main 위에 올리기
git switch rebase-practice
git rebase main
```

```
# 충돌이 나면:
# 1. 파일 수정
# 2. git add .
# 3. git rebase --continue

# 취소하고 싶으면:
# git rebase --abort
```

---

## 파일 구조

```
hands-on/
├── README.md              ← 이 파일 (실습 가이드)
├── src/
│   ├── hello.py           ← 실습 1, 5용
│   ├── greeting.py        ← 실습 2용 (핵심 충돌 실습)
│   ├── utils.py           ← 실습 3용 (abort 실습)
│   └── calculator.py      ← 실습 4용 (squash 실습)
└── tests/
    └── test_greeting.py   ← greeting.py 테스트 (선택)
```

## 팁

- 막히면 `git status` 를 먼저 확인하세요
- 충돌 마커( `<<<` , `===` , `>>>` )는 반드시 모두 지워야 합니다
- VS Code를 쓰면 "Accept Current / Incoming / Both" 버튼으로 쉽게 해결할 수 있습니다
- 모든 실습이 끝나면 `git log --oneline --graph --all` 로 전체 히스토리를 확인해 보세요